

HOOGLY ENGINEERING & TECHNOLOGY COLLEGE



Final Year Project (2022-2026)

DTrain – Decentralized Machine Learning Training Platform (PaaS)

Web Development

TEAM MEMBERS

Name	University Roll No
Debjit Mitra	17600122046
Soham De	17600122062
Akash Poddar	17600122071
Sourodip Ghosh	17600122040

Mentored by Assistant Professor Agnimita Banerjee

Problem Statement

Progress in applied machine learning is limited less by algorithms or talent than by a lack of affordable, reliable, low-friction compute — especially for independent researchers, students, and early-stage practitioners in cost-sensitive economies like India. The problem statement quantifies this cost barrier, outlines the structural pain points shown below in the current ML training ecosystem, and shows why existing alternatives fail to solve them together.

The Challenge:

Machine learning compute is expensive and out of reach



Cost & Inflexibility

Centralized cloud platforms charge premium, fixed hourly rates — wasting spend whenever GPUs sit under-utilised.



Untapped Potential

A vast, globally distributed pool of powerful idle GPUs (gaming rigs, workstations) sits unused every day.



Barrier for Learners

Students, researchers and indie developers without a high-end PC or cloud budget are locked out of serious training.

Facts & Figures Behind The Problem

Machine learning training has a compute problem. Cloud GPUs are expensive, hardware is getting pricier every year, and most learners simply don't have access to the horsepower they need.

\$2–10/hr

Cloud GPU pricing — a mid-sized run can cost ₹15,000–₹50,000

200%

Rise in GPU hardware prices, 2020 to 2024

2×

Growth in AI compute demand every six months

70%

ML students say hardware limits stop effective training

45%

Companies report compute-related project delays

10–15%

ML learners with a dedicated GPU — rest rely on laptops, 10–20× slower

30–40%

Extra long-term spend, locked into one cloud provider

80%+

Learners with no GPU access, stretching training to days

Our Solution

DTrain works like a marketplace, not a subscription. Once a user tops up their wallet, funds are reserved the moment a job is published — but nothing is actually charged until training finishes successfully. This protects both sides: users only pay for work that's actually completed, and workers are guaranteed payment once they deliver.

1

Upload

User uploads a ZIP with training code, data & requirements.txt.

2

Reserve & Publish

User tops up their wallet and publishes the job — funds are reserved, not charged yet.

3

Train

A free worker node trains the model in an isolated Docker container.

4

Download & Pay

Training finishes, user downloads the model, and payment is deducted only now — from the reserved balance.

The result: No cloud accounts, no infrastructure setup, and you're only charged once your model is actually trained.

Core Architecture

D-Train is built from four cooperating components that together turn an idle GPU into a usable training node. A user-facing web dashboard handles job uploads, pricing, and payments, while a backend API server orchestrates everything behind the scenes — authentication, job matching, and billing. On the worker side, an Electron Worker dashboard gives GPU owners visibility into incoming jobs. It runs quietly in the background, pulling jobs, spinning up Docker containers, and streaming logs back in real time, and Dockerized containers where the isolated model training happens.



User Web Dashboard

Where trainers upload jobs, review AI-quoted pricing, top up their wallet, and download finished models.



Backend API Server

The central hub — auth, job orchestration, the pricing engine, and payment settlement all run here.



Electron Worker

Browser interface for GPU owners to view incoming jobs, track earnings, and monitor node status.



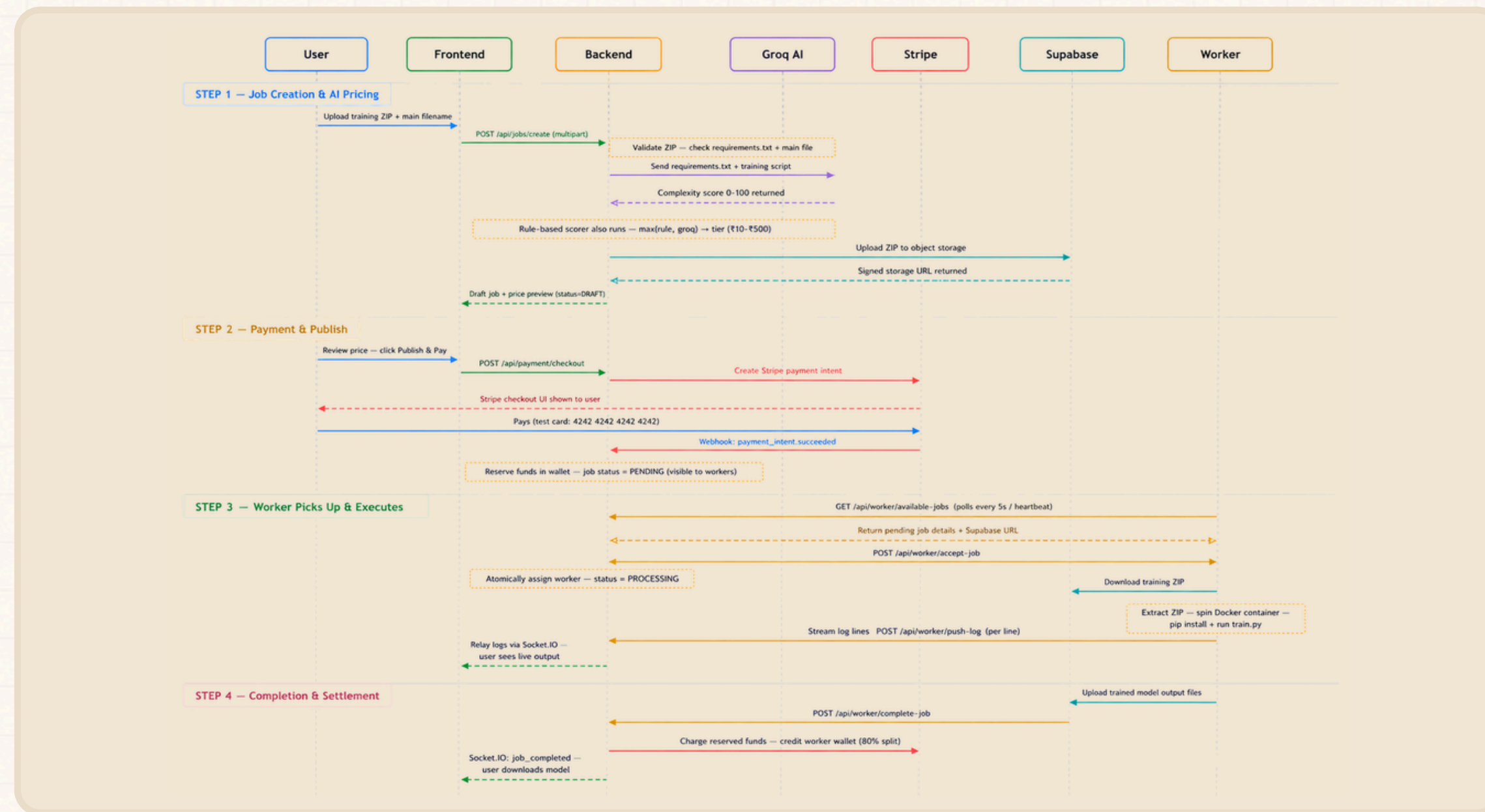
Dockerized Containers

Runs the models in an isolated Docker environment, streaming logs in real time.

How It Flows

End-to-end workflow (sequence):

From ZIP upload to a downloaded, trained model - every hand-off between user, API, worker and storage.

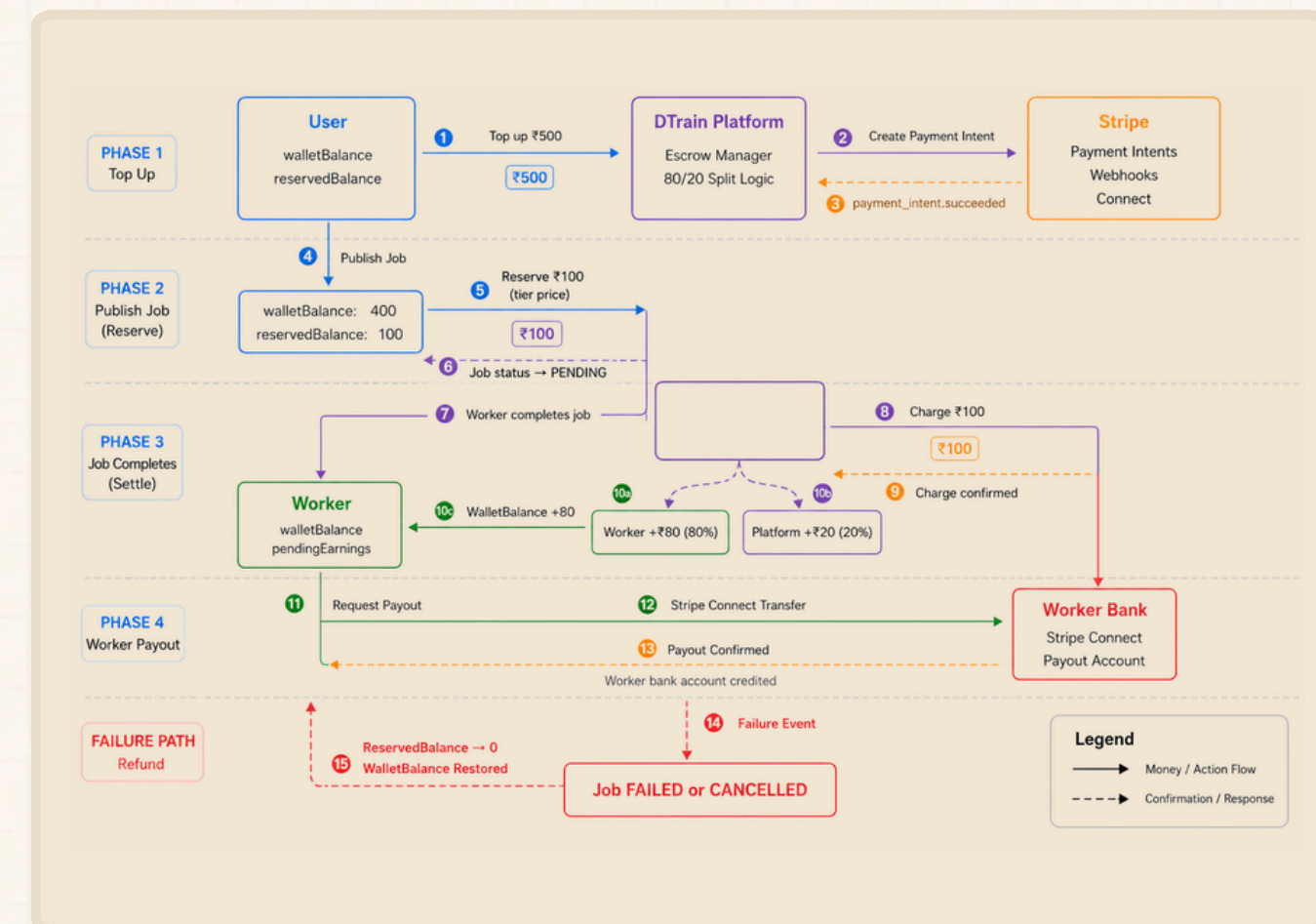


Payment & wallet system

Every job on DTrain is priced automatically and paid for safely. An AI-assisted engine scores each job's complexity and sets a fair flat price — then a reserve-and-charge system built on Stripe makes sure no one pays until training actually finishes.

- 1 Wallet top-up** User adds funds via a Stripe payment intent.
- 2 Publish → Reserve** Publishing a job reserves funds — nothing charged yet.
- 3 Worker accepts** The job is locked to a single worker.
- 4a Success → Charge** Charged: worker +80%, platform +20%.
- 4b Failure → Refund** Reservation released; full refund to wallet.

Workers can request a payout to their bank account via Stripe Connect any time.

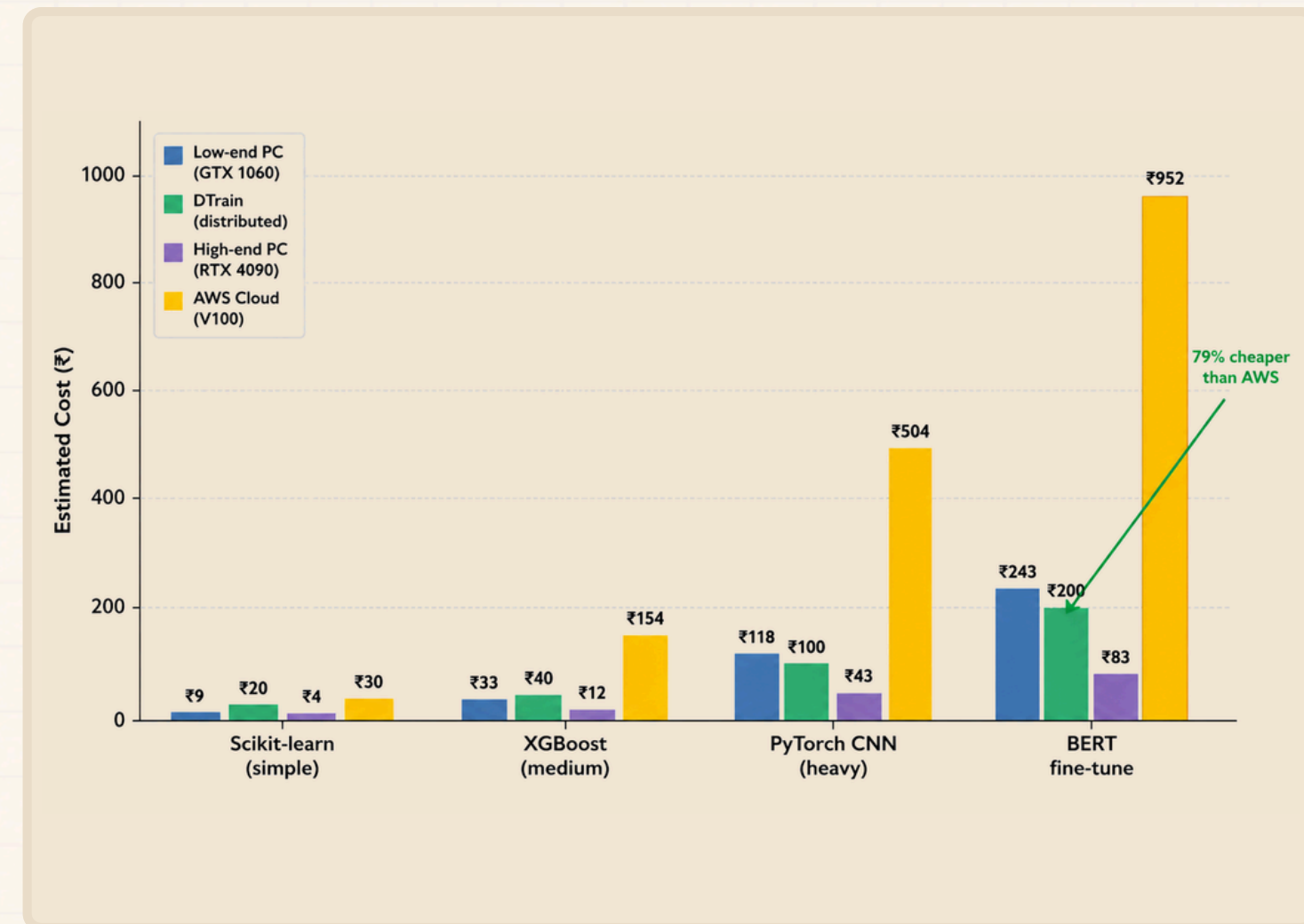
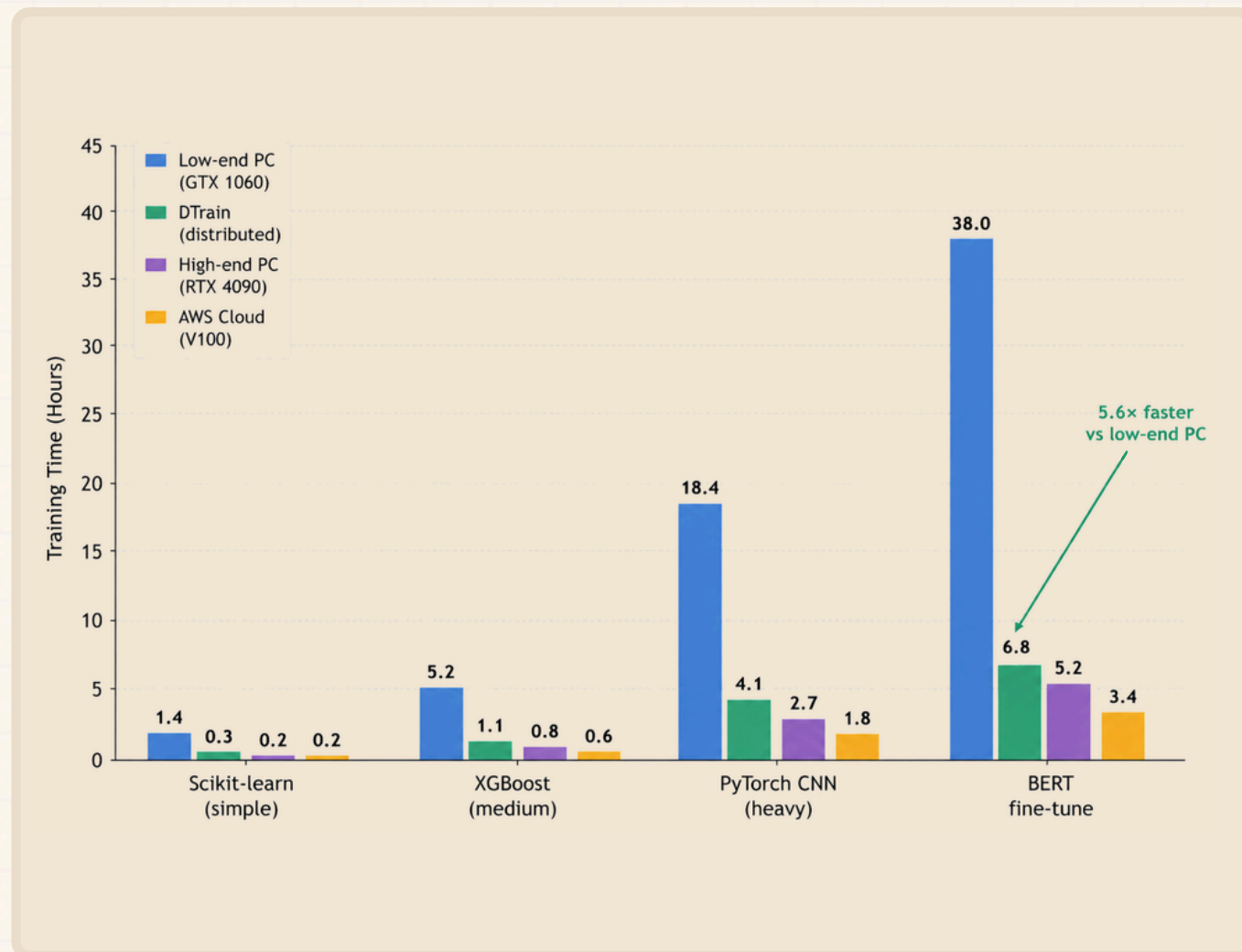


Technology Used

Component	Technology	Purpose
Frontend	React 18, TypeScript, Vite, Tailwind	Clean, responsive user dashboard
Backend / API	Node.js, Express v5, Socket.IO	REST API, auth, orchestration, log relay
Database	MongoDB (Atlas) + Mongoose	Jobs, users, workers, billing
Job Queue	Redis (ioredis) Pub/Sub	Instant job dispatch & coordination
Object Storage	Supabase Storage	ZIP uploads & trained model files
Worker Agent	Electron + Node.js	Cross-platform desktop worker app
Execution	Docker	Isolated, reproducible training runs
AI Pricing	Groq LLM API	Job complexity scoring for pricing
Payments	Stripe + Stripe Connect	Escrow, top-up, automatic payouts
Authentication	JWT + bcryptjs	Stateless auth, secure password hashing

Performance & Cost – A Modeled Comparison

Training time and cost were calculated using published GPU specs (TFLOPS) and real AWS pricing for each hardware tier, since testing identical jobs across four physical machines wasn't feasible within project scope. These are projected estimates, not numbers measured on live hardware.

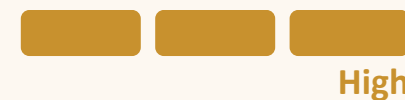


Feasibility Study

DTrain is technically sound, built on proven open-source tools. It's economically lean too, with no fixed infrastructure cost. The main constraint is operational — needing enough active workers for fast job pickup. The core platform was fully built and tested within our two-semester timeline.



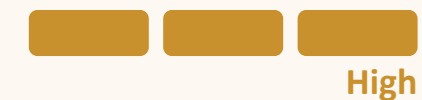
Technical



Built entirely on proven, well-documented, mostly open-source tools — Node.js, MongoDB, Redis, Docker, React — with strong community support and no exotic dependencies.



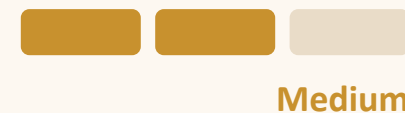
Economic



No fixed infrastructure cost — workers use their own hardware. The platform earns a 20% cut per completed job, so cost scales with usage, not upfront investment.



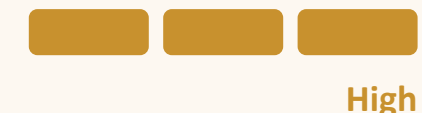
Operational



Needs a critical mass of active worker nodes for fast job pickup. Cold-start liquidity is the main real-world constraint until the network grows.



Schedule



Core platform — pricing, payments, worker orchestration — was designed, built and tested inside the two-semester project timeline using rapid-development frameworks.

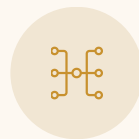
Future Work & Roadmap

DTrain works today, with room to grow — GPU-accelerated containers, multi-worker training, a worker reputation system, macOS/Linux support, job templates, and an admin analytics dashboard are all next.



GPU-Accelerated Containers

Add NVIDIA container toolkit support so Docker jobs can use worker GPUs directly, not just CPU.



Multi-Worker Parallel Training

Split large jobs across several workers at once to cut training time further for heavy models.



Worker Reputation System

Rate and rank workers on speed and reliability, so job creators can prefer trusted nodes.



macOS & Linux Agents

Extend the Electron worker app beyond Windows so any machine can join the network.



Job Template Library

Ready-made starter templates for common ML tasks, so users can launch jobs faster.



Admin Analytics Dashboard

Platform-wide visibility into job volume, worker health, and revenue for operators.

Conclusion

DTrain exists to close a simple but real gap – the people who most need to train machine learning models, students, researchers, and independent developers, are usually the ones least able to afford the compute to do it. By turning idle GPUs sitting unused around the world into a shared, pay-as-you-go resource, DTrain makes serious model training accessible to anyone with an idea worth training, not just those who can afford a high-end PC or a cloud subscription.

More than a cost saving, this is a shift in who gets to participate in AI. A student without a GPU, a researcher without a lab budget, or a developer testing an idea on the side can now train a real model for a few rupees instead of being priced out entirely. In doing so, DTrain points toward a future where compute is treated less like a gatekeeper and more like a shared community resource – one job, one worker, one training run at a time.



THANK YOU